

Fases de experimentacion

VULNERABILIDAD EN LA REFACTORIZACIÓN DE CÓDIGO GENERADO POR IA

Josue Chango
Yostin Sisalema



Fases

- Introducción: Se define el problema, describe el objetivo el proposito y sobretodo el contexto, junto a las preguntas de investigacion a las que se van a responder
- Diseño y Planificación: En esta seccion se presenta el objetivo general junto con la hipótesis las variables dependientes he independientes respectivamente y se realiza un diseño para el experimento
- Ejecucion y recoleccion de datos: Se realiza un protocolo de ejecucion y se validan los datos obtenidos
- Analisis e interpretacion: Se realiza un analisis estadistico y se observan los resultados obtenidos
- Reporte de resultados: Se realiza una discusion en donde se tratan temas como la relevancia para la industria, las amenazas a la validez y sobre el trabajo futuro

Contexto del problema

Problema:

No se sabe qué pasa con las vulnerabilidades de seguridad del código generado por LLMs una vez que un desarrollador humano lo refactoriza y lo prepara para producción. La refactorización manual, ¿elimina esas debilidades, las deja intactas, o incluso introduce nuevas? Y ¿cómo se compara ese resultado con usar herramientas de IA especializadas en seguridad para hacer esa misma refactorización?

Contexto:

- Es bien sabido que los LLMs generan código con fallas de seguridad conocidas (CWE Top 25) de forma habitual, inyección de comandos (CWE-78), inyección SQL (CWE-89), entre otras— incluso en modelos avanzados como GPT-4o.
- CodeEnhancer (Lee et al., 2026) es el sistema que estableció este hallazgo, combinando LLMs con herramientas SAST (análisis estático) para detectar esas debilidades.

Objetivo general (GQM)

Analizar la refactorización de código Python generado por LLMs para comparar la cantidad de vulnerabilidades mantenidas o introducidas entre refactorización manual y asistida por IA con SAST, desde la perspectiva de equipos de desarrollo y estudiantes avanzados.



Preguntas de investigación

RQ1

¿Cuántas vulnerabilidades por categoría CWE permanecen tras refactorización manual vs asistida por IA?

RQ2

¿Hay diferencias en precisión y recall al identificar y corregir vulnerabilidades entre ambos grupos?

RQ3

¿El efecto de la asistencia de IA varía entre fallos sintácticos (patrón) y fallos lógicos/estructurales?

Hipótesis y nivel de significancia

1

H_0

No existe una diferencia estadísticamente significativa en la cantidad de vulnerabilidades de seguridad introducidas o mantenidas entre el código refactorizado manualmente y el código refactorizado con asistencia de sistemas de IA orientados a seguridad.

$$\alpha = 0.05$$

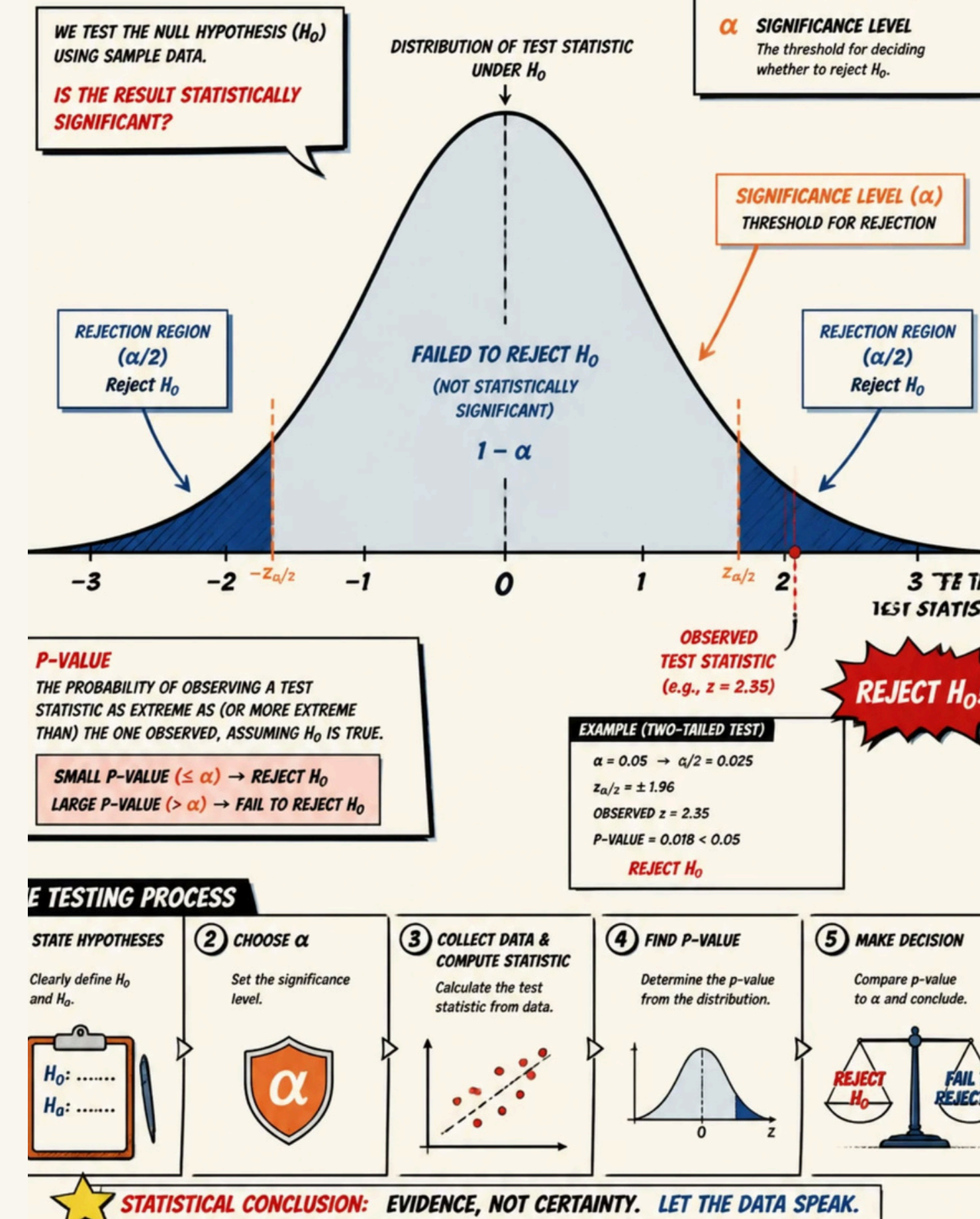
2

H_1

Existe una diferencia estadísticamente significativa entre las condiciones.

HYPOTHESIS TESTING

VISUAL GUIDE TO STATISTICAL INFERENCE



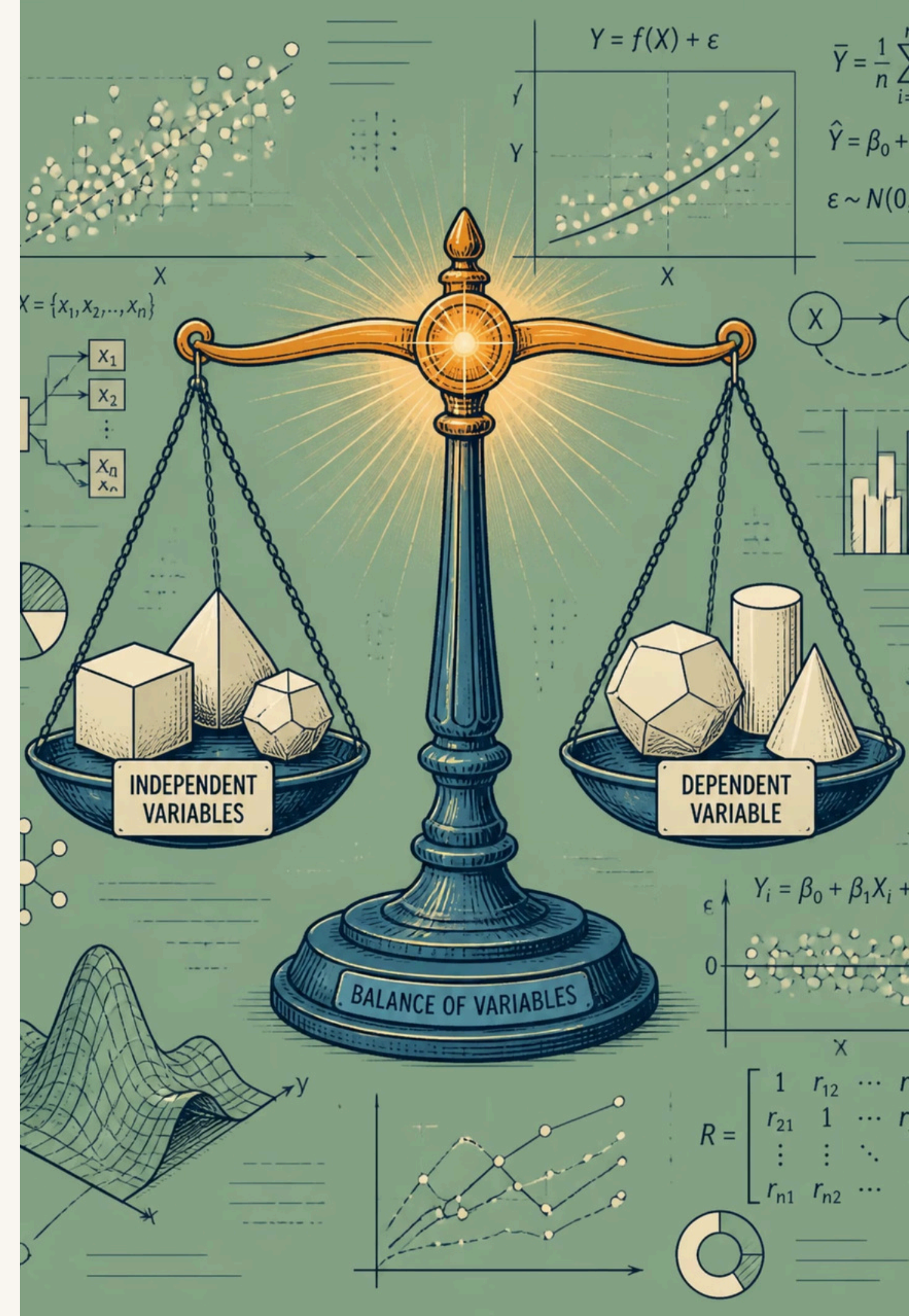
Variables del experimento

Independiente

Modalidad de refactorización:
manual vs asistida por IA con
SAST.

Dependiente

Número de vulnerabilidades por
categoría CWE antes y después
de refactorizar.



Diseño experimental e instrumentación

Diseño A/B con dos grupos independientes: control (manual) y experimental (IA + SAST). Asignación aleatoria de participantes y muestras, balanceando por experiencia.

- Modelos generadores: GPT-4o y Claude-3.5 Sonnet (prompt Step-by-Step).
- Herramientas SAST: Bandit, Pylint; complementos: SonarQube, PMD.



Métricas y validación

→ Detección

Precisión, recall y F1 por generador y modalidad.

→ Calidad

Tasa de falsos positivos (FPR) y tasa de éxito de reparación estática (SonarQube).

→ Impacto

Reducción porcentual de fragmentos vulnerables antes/después.

Registro de vulnerabilidad solo si coinciden ≥ 2 detectores o detector + revisión humana.

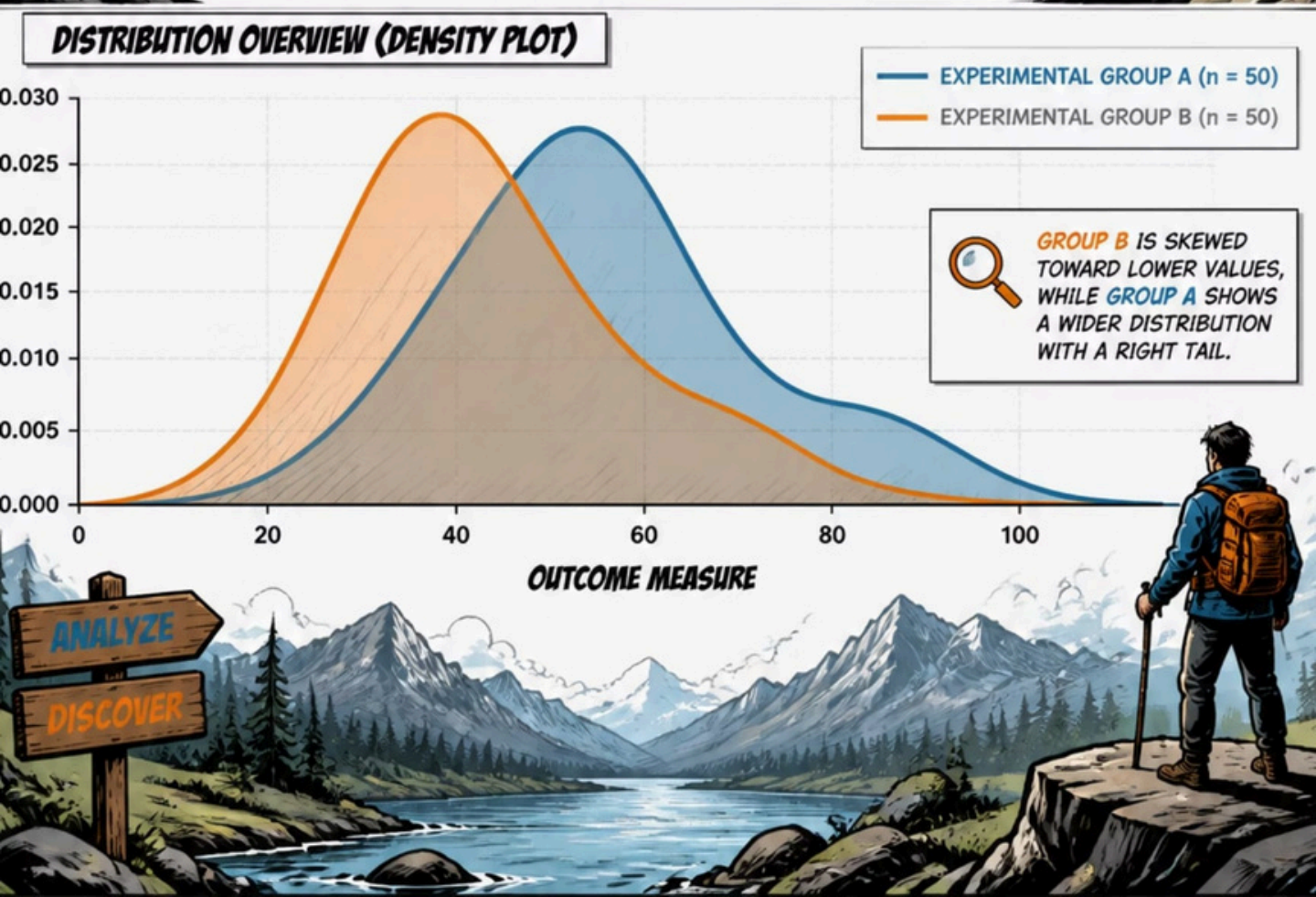
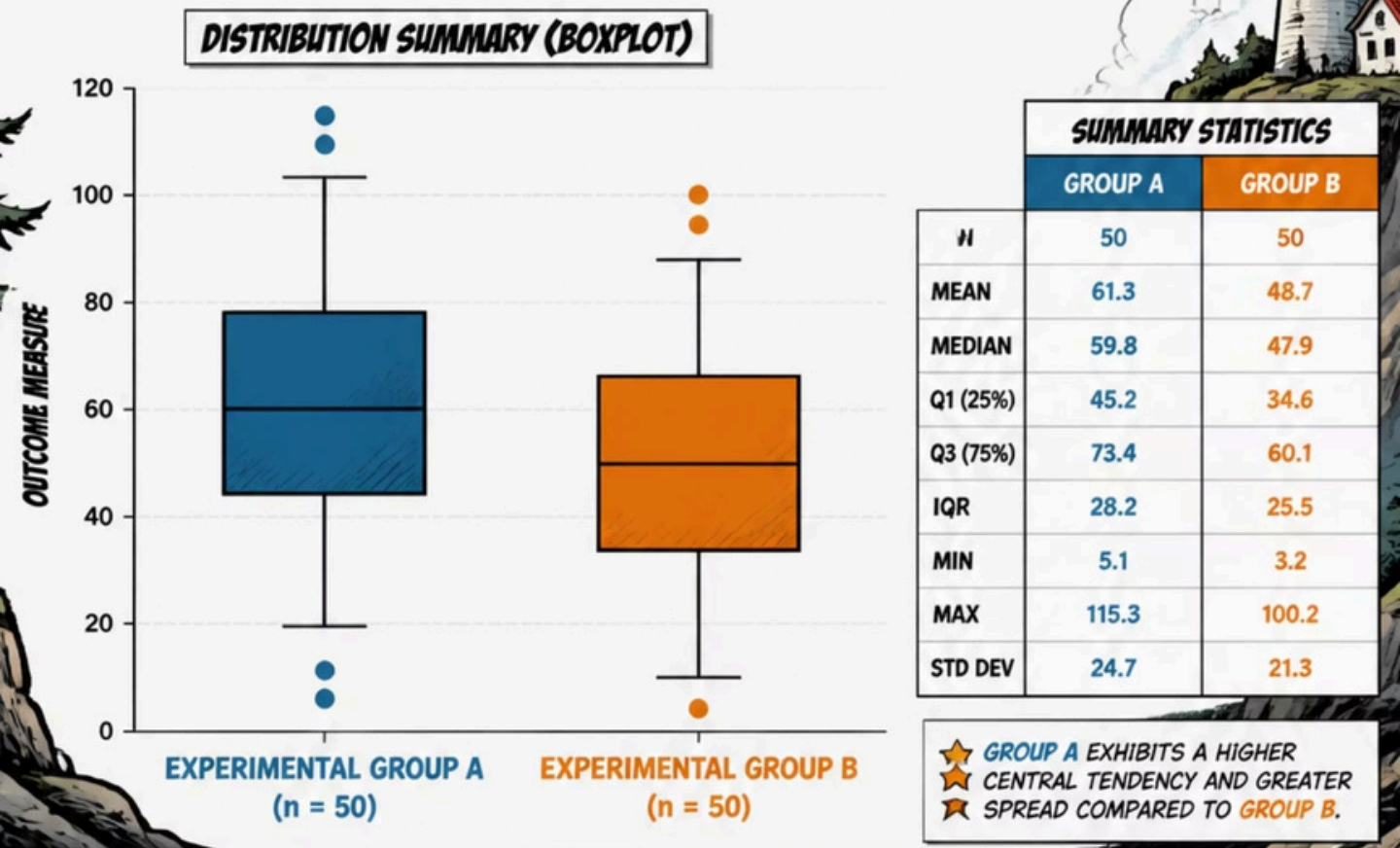


Protocolo de ejecución

1. Generar muestras con GPT-4o y Claude-3.5 (Step-by-Step).
2. Escaneo SAST inicial (Bandit, Pylint, SonarQube).
3. Asignación aleatoria de participantes y muestras, balanceo por experiencia.
4. Refactorización manual (sin herramientas automáticas).
5. Refactorización asistida: ciclo LLM + feedback SAST hasta convergencia o límite de iteraciones.
6. Escaneo SAST posterior y verificación manual antes de calificar.

COMPARATIVE STUDY: EXPERIMENTAL GROUP A vs. EXPERIMENTAL GROUP B

A COMPARISON OF OUTCOME MEASURES BETWEEN TWO EXPERIMENTAL GROUPS TO ASSESS DISTRIBUTION, CENTRAL TENDENCY, AND VARIABILITY.



Análisis esperado y resultados anticipados

Para esta parte primero se calcula la estadística descriptiva (media, mediana, moda, etc) y a partir de esto se compara graficos para pasar a la deduccion y la conclusion

En el caso de nuestro contexto: Si comparamos más de dos configuraciones (por ejemplo, distintas combinaciones de SAST y LLM), aplicaremos la prueba post hoc de Nemenyi (rangos medios). Si en cambio optamos por un diseño de medidas repetidas, donde la misma persona refactoriza en ambas condiciones sobre muestras equivalentes, usaremos la prueba t pareada

Aqui se espera que la condición asistida por IA reduzca significativamente vulnerabilidades totales, con efecto más fuerte en fallos de patrón sintáctico que en fallos lógicos/estructurales (p.ej. CWE-89, CWE-78)

Discusión, amenazas y trabajo futuro

Riesgos

Validez interna: diferencia de habilidad mitigada por aleatorización y balanceo.

Validez externa: limitación a Python y subconjunto CWE.

Constructo: errores de SAST mitigados por doble confirmación.

Futuro

- Réplicas a mayor escala y otros lenguajes.
- Comparar múltiples familias de LLM.
- Crear un banco de pruebas específico para refactorización.

